Modélisation des transferts d'énergie au sein d'une aurore boréale

Matei Cristinar



Introduction

Problématique : Comment modéliser les échanges énergétiques qui ont lieu durant une aurore boréale ?



Plan

I. Présentation du phénomène général

II. Modélisation

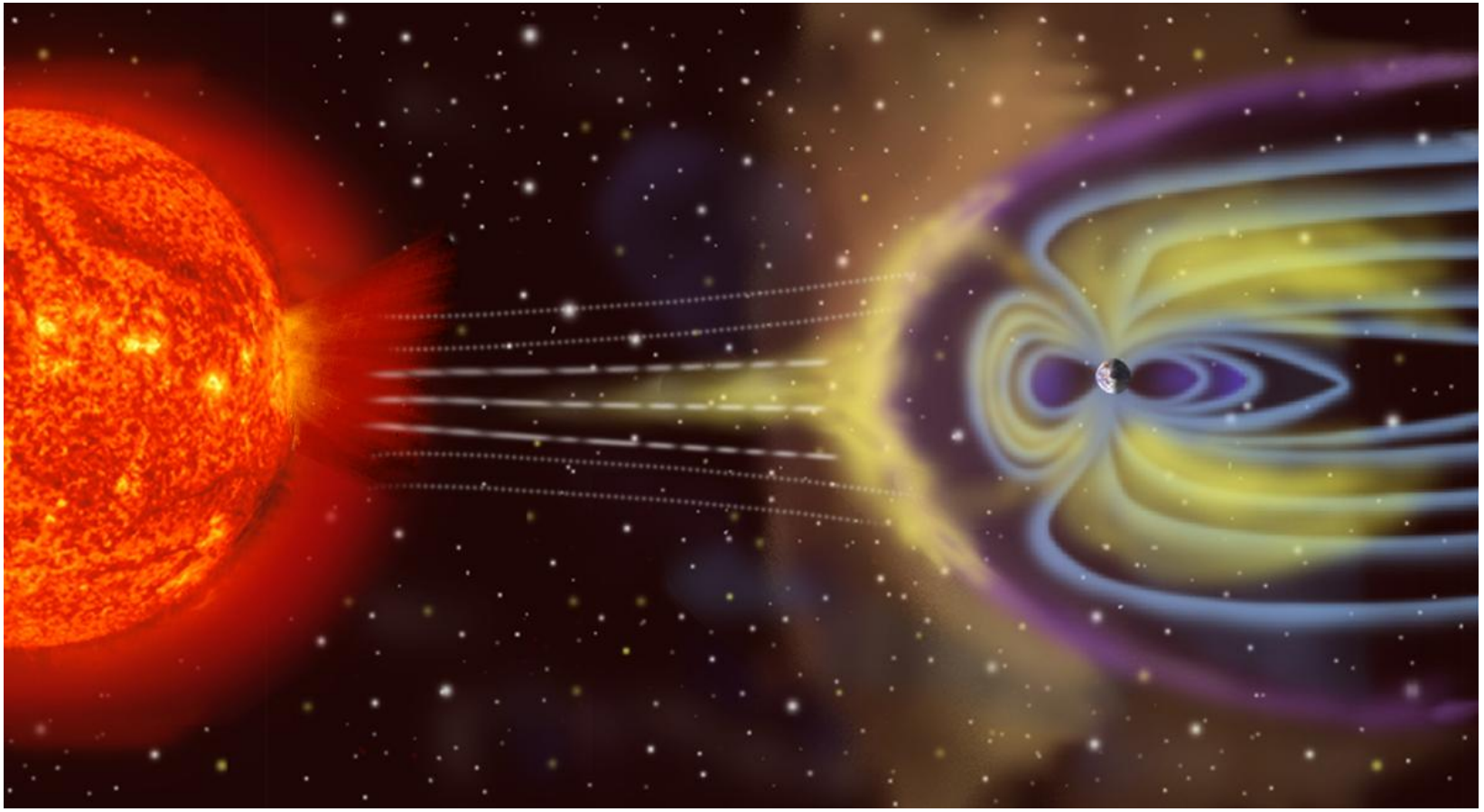
III. Exploitation des résultats

I. Présentation du phénomène général

II. Modélisation

III. Exploitation des résultats

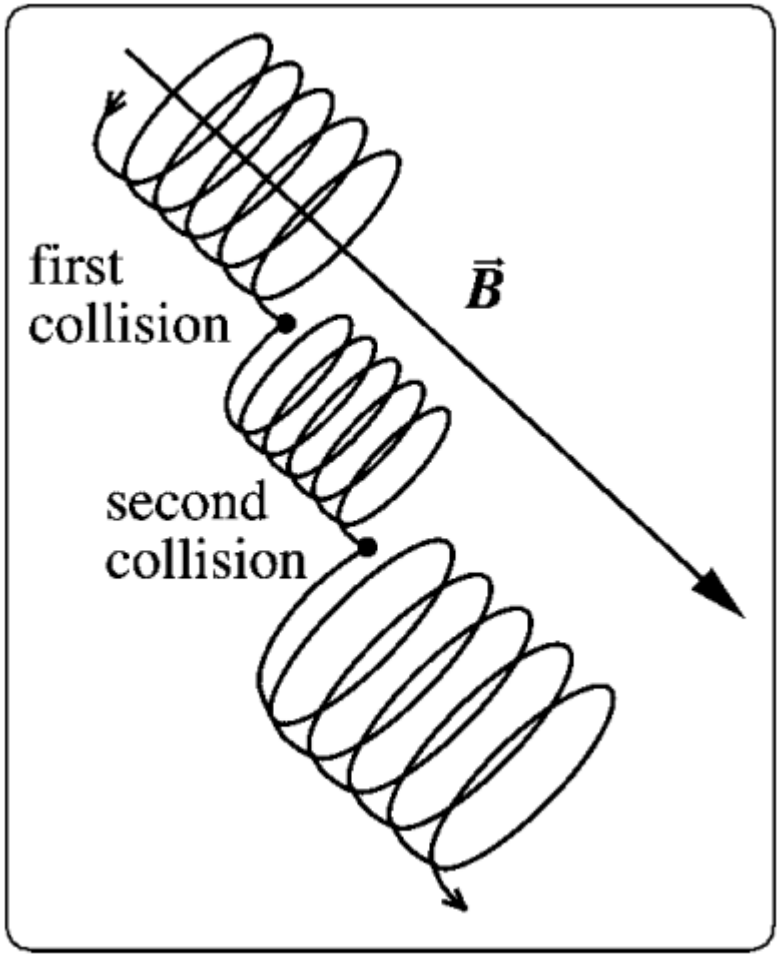
Naissance des aurores



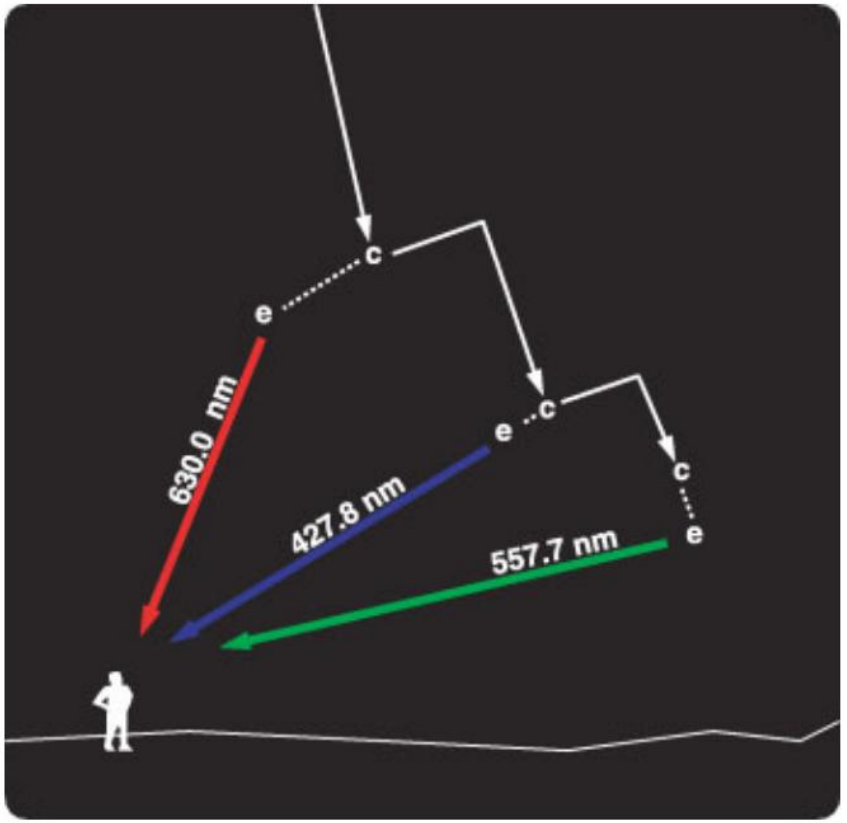
Source : The European Space Agency

Présentation du phénomène

Trajet de l'électron dans l'atmosphère



Collisions qui provoquent les aurores



Source : Simulating the aurora

Modélisation

I. Présentation du phénomène général

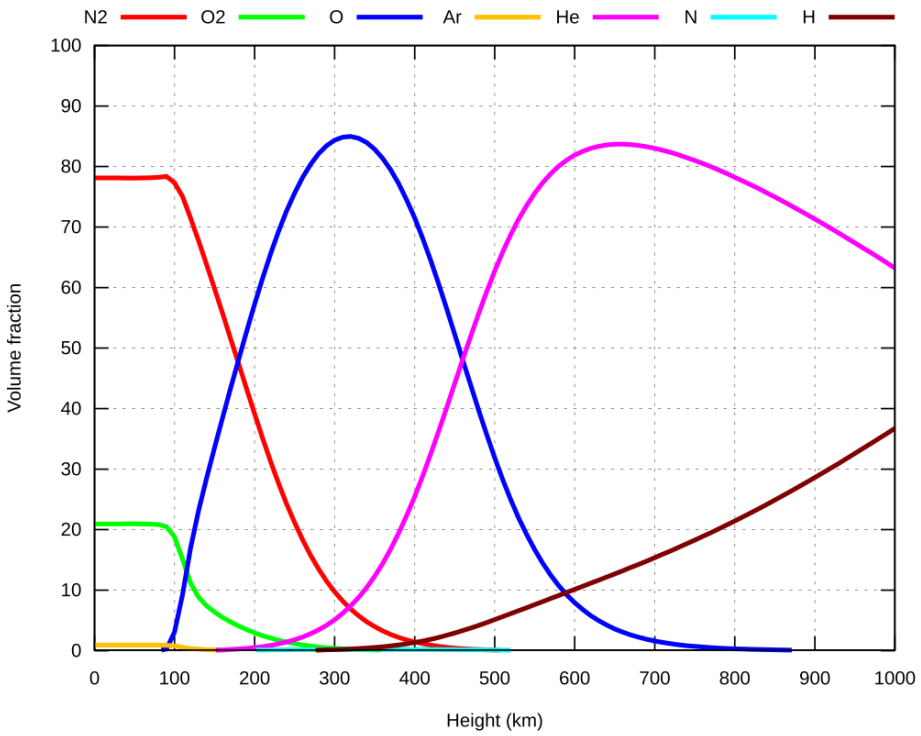
II. Modélisation :

- Atmosphère
- Collisions
- Type de collision

III. Exploitation des résultats

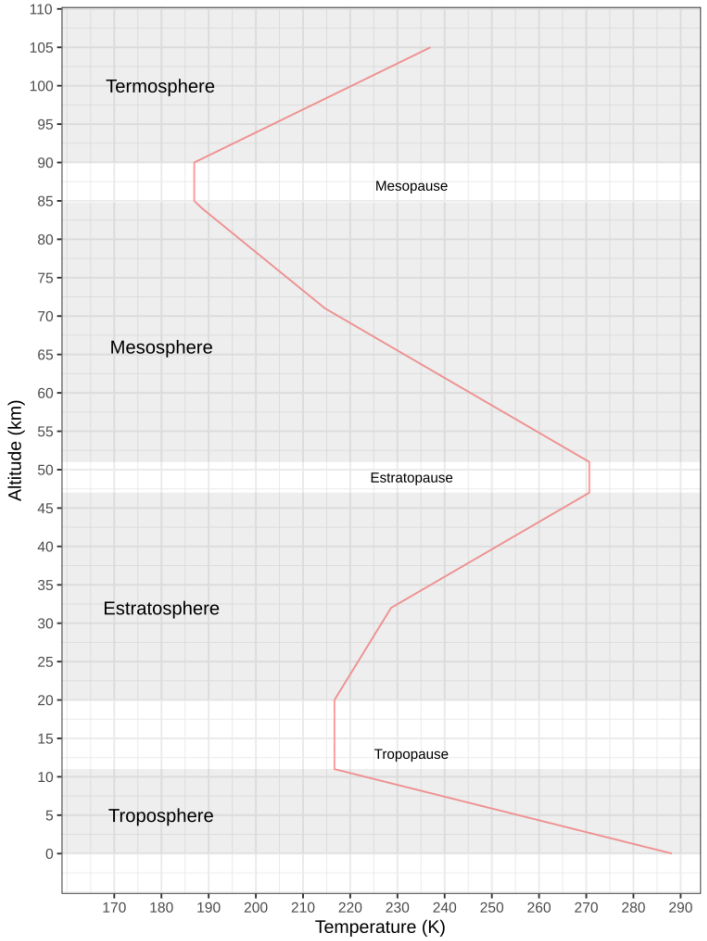
Modélisation de l'atmosphère

Composition atmosphérique



Source : MSIS-E90 model

Température atmosphérique



Source : US Standart atmosphere level

Recherche de l'expression de la pression

L'équation hydrostatique :

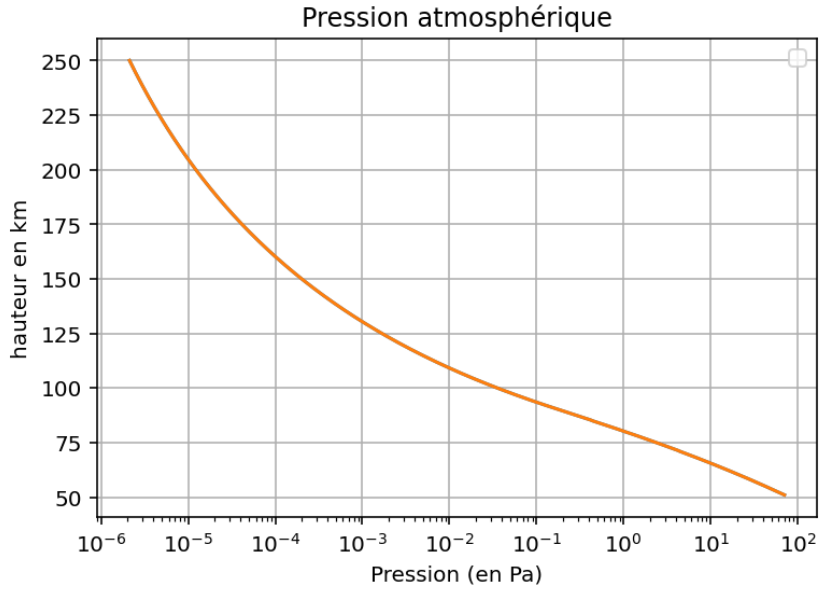
$$\frac{\partial P}{\partial z} = -\rho g$$

L'équation des gaz parfaits :

$$\rho = \frac{Pm}{k_B T}$$

$\Rightarrow$

$$P(z) = P(z_0) \cdot \exp\left(-\int_{z_0}^{z} \frac{m(h) \cdot g(h)}{k_B T(h)}\, dh\right)$$

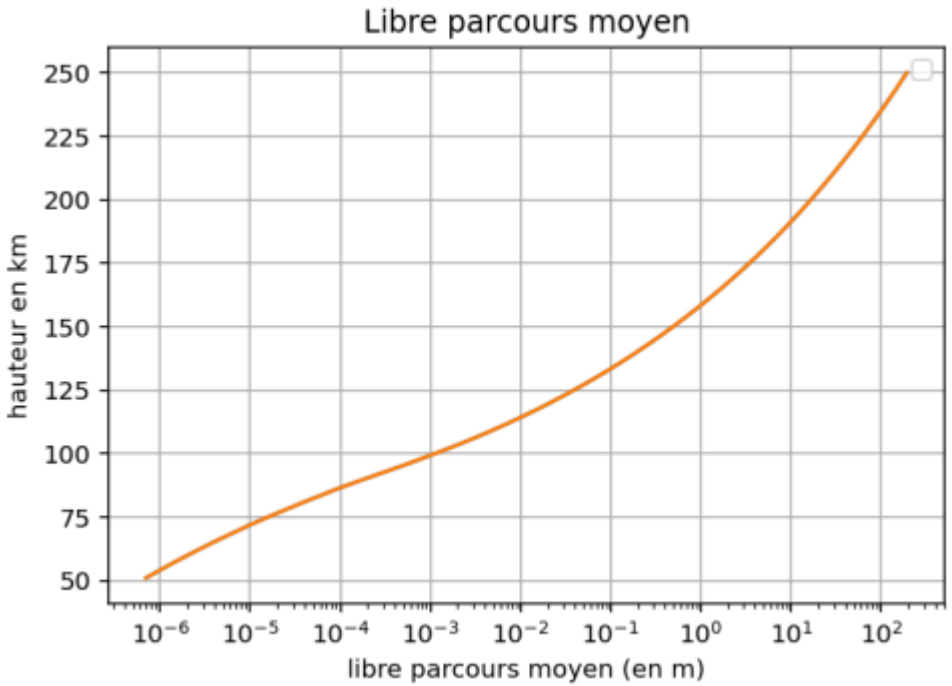


Obtention du libre parcours moyen

Expression du libre parcours moyen :

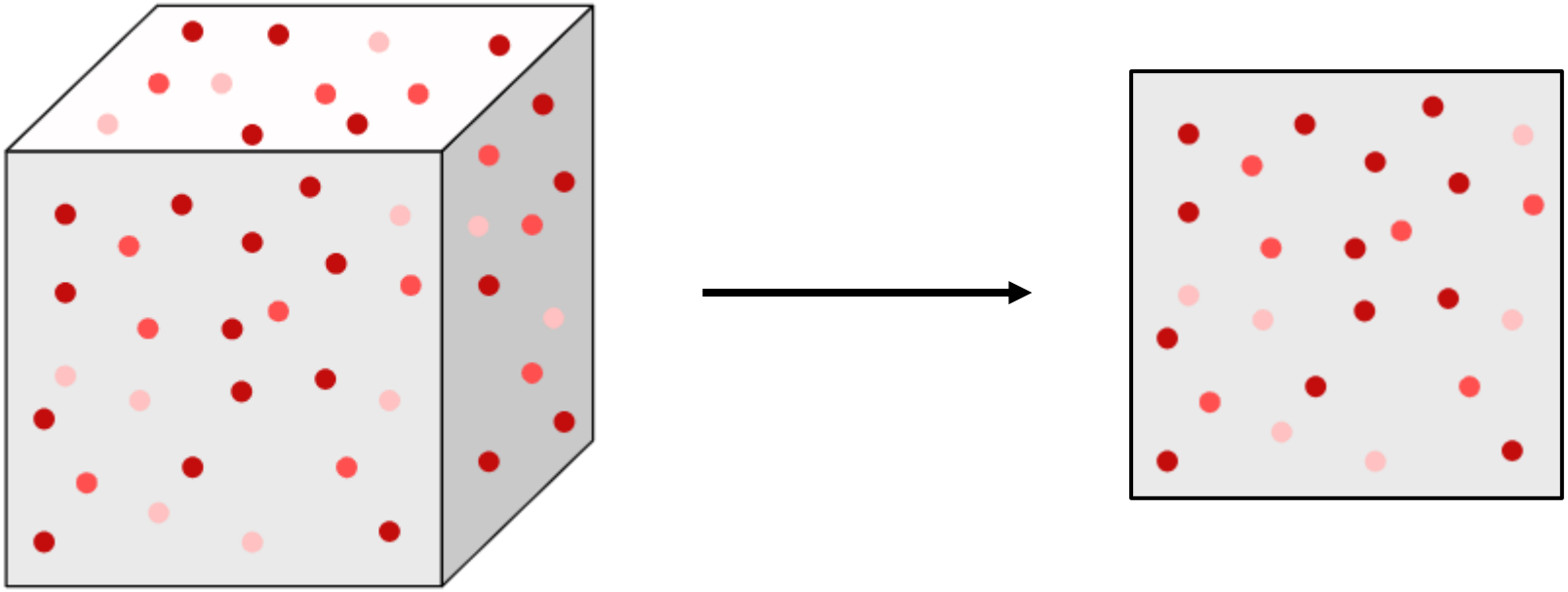
$$\lambda = \frac{k_B T}{\pi d^2 P}$$

$\Rightarrow$



D est le rayon effectif des molécules a une certaine hauteur de l'atmosphère

Première approche : utilisation de la concentration particulaire



Source : nagwa.com

Première approche : utilisation de la concentration particulaire

Dans ce cas :

$$P_{coll} = \frac{V_{occupé}}{V_{total}} = n(h) \cdot \frac{4}{3}\pi r_{eff}^3$$

Problème : cette approche naïve demande trop de ressources pour le calcul des collisions

Deuxième approche : utilisation d'une loi de Poisson

Caractérisation du processus de Poisson :

I. Le nombre d'occurrence dans des intervalles l disjoints sont indépendants.

II. La probabilité d'une occurrence dans un petit intervalle l est de $\mu \cdot l$.

III. La probabilité d'une occurrence dans un petit intervalle est négligeable

Deuxième approche : utilisation d'une loi de Poisson

Le nombre de collisions dans un intervalle l est modélisé par la variable aléatoire N_l *, et* N_l *suit une loi de Poisson de paramètre* μ.

Ici, $\mu = \frac{l}{\lambda}$ *car on a :*

$$P(N_{l+a} - N_l) = \frac{a}{\lambda} + o(a)$$

Deuxième approche : utilisation d'une loi de Poisson

On obtient donc :

$$P\left(N_l = k\right) = e^{-l/\lambda}\frac{\left(l/\lambda\right)^k}{k!}$$

On fixe la distance parcourue à chaque itération : $\mathrm{l} = \max\left(\lambda,5\right)$

Deuxième approche :

Combien il y a-t-il de collisions à chaque itération ?

$$P\,(N_l = 0) = e^{-1} = 36\%$$

$$P\,(N_l = 1) = e^{-1} = 36\%$$

$$P\,(N_l = 2) = \frac{e^{-1}}{2!} = 18\%$$

$$P\,(N_l = 3) = \frac{e^{-1}}{3!} = 6{,}1\%$$

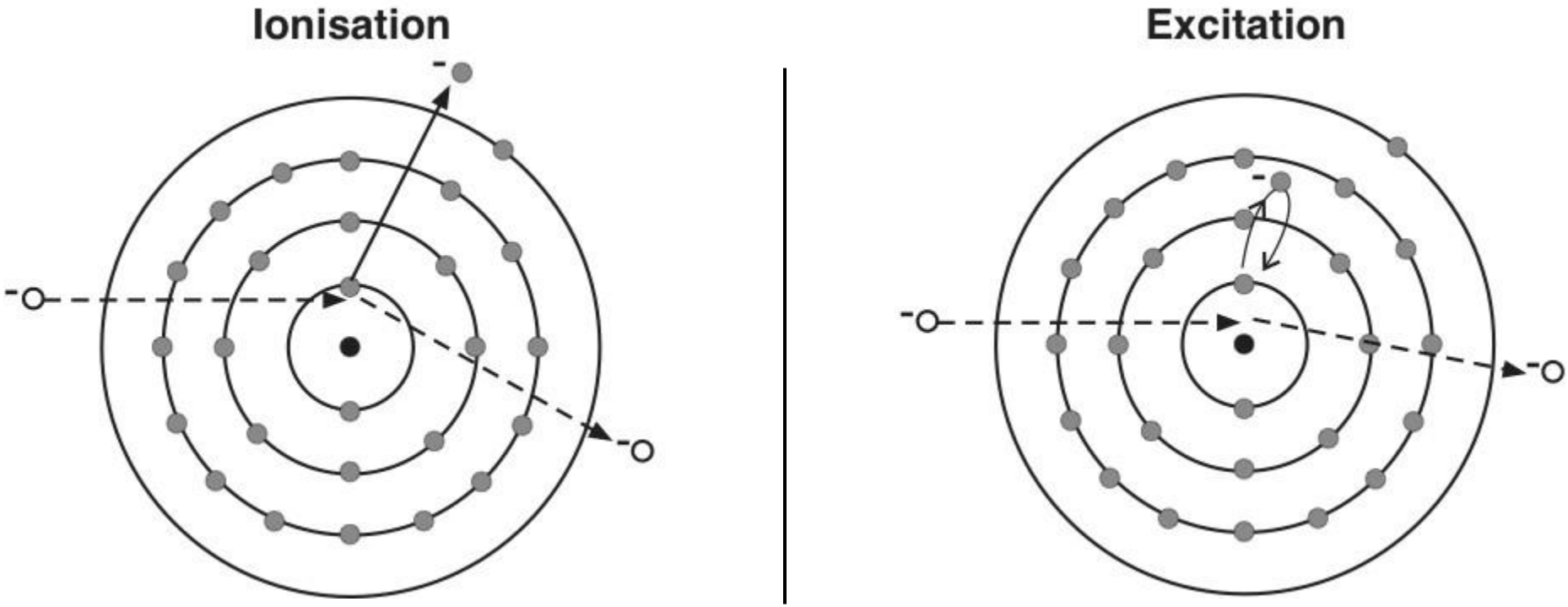
$$P\,(N_l = 4) = \frac{e^{-1}}{4!} = 1{,}5\%$$

$$P\,(N_l = 5) = \frac{e^{-1}}{5!} = 0{,}3\%$$

$\Longrightarrow$

Seulement 4 collisions maximum

Modélisation des collisions



Source : Radioprotection en radiodiagnostic – 3e édition

Modélisation des collisions

Ionisation

Si $E_{electron} \geq E_{ionisation}$

Excitation

Si $E_{excitation} \leq E_{electron} \leq E_{ionisation}$

Modélisation globale

Température et composition atmosphérique

L'équilibre hydrostatique, modèle des sphères dures



Pression puis libre parcours moyen

Modélisation globale

A une hauteur h = 250 km,
et énergie Ei = 5 keV

libre parcours moyen

Processus de Poisson



Probabilité qu'il y ait une collision

Modélisation globale

En cas de collision → Ionisation

↓

Excitation

$$E_f = E_i - E_{coll}$$
$$h_f = h_i - \max(\lambda(h_i), 5)$$

I. Présentation du phénomène général

II. Modélisation

III. Exploitation des résultats

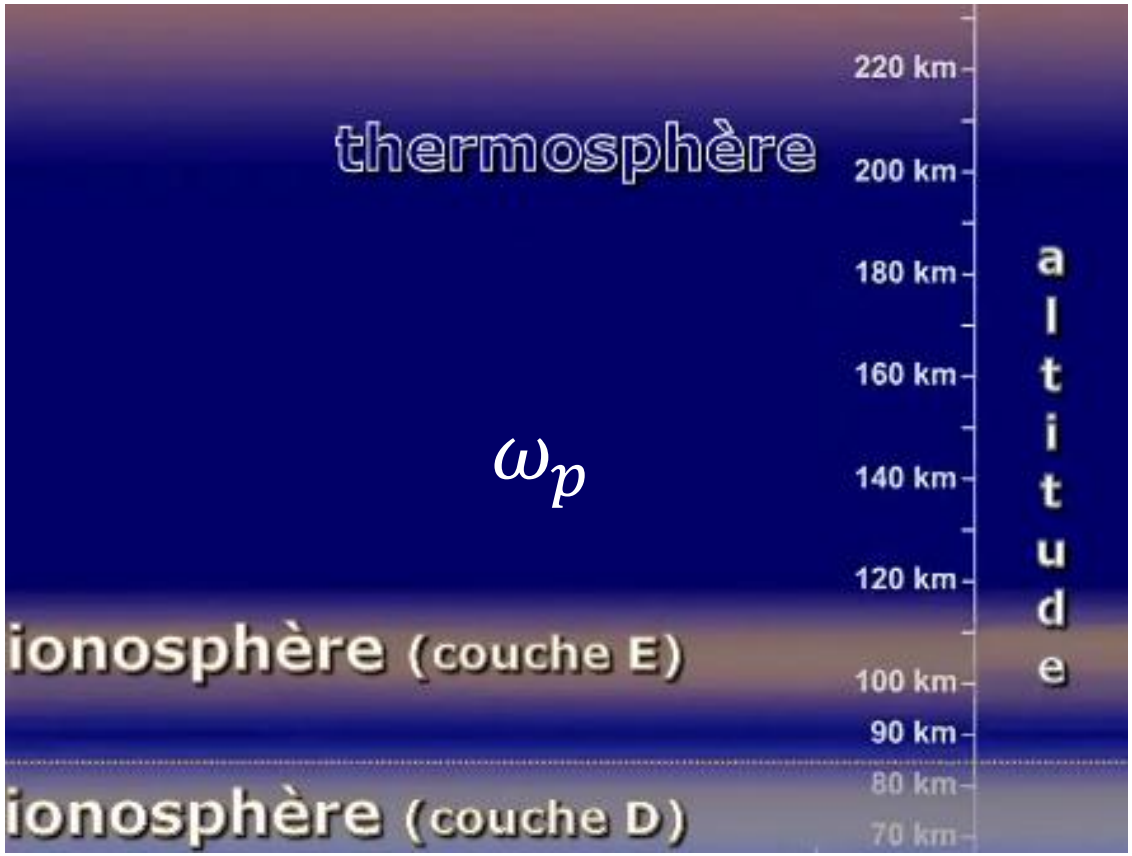
Conséquences

↗ de la concentration particulaire des ions et électrons dans la ionosphère

$\Rightarrow$ Modification des propriétés de la ionosphère

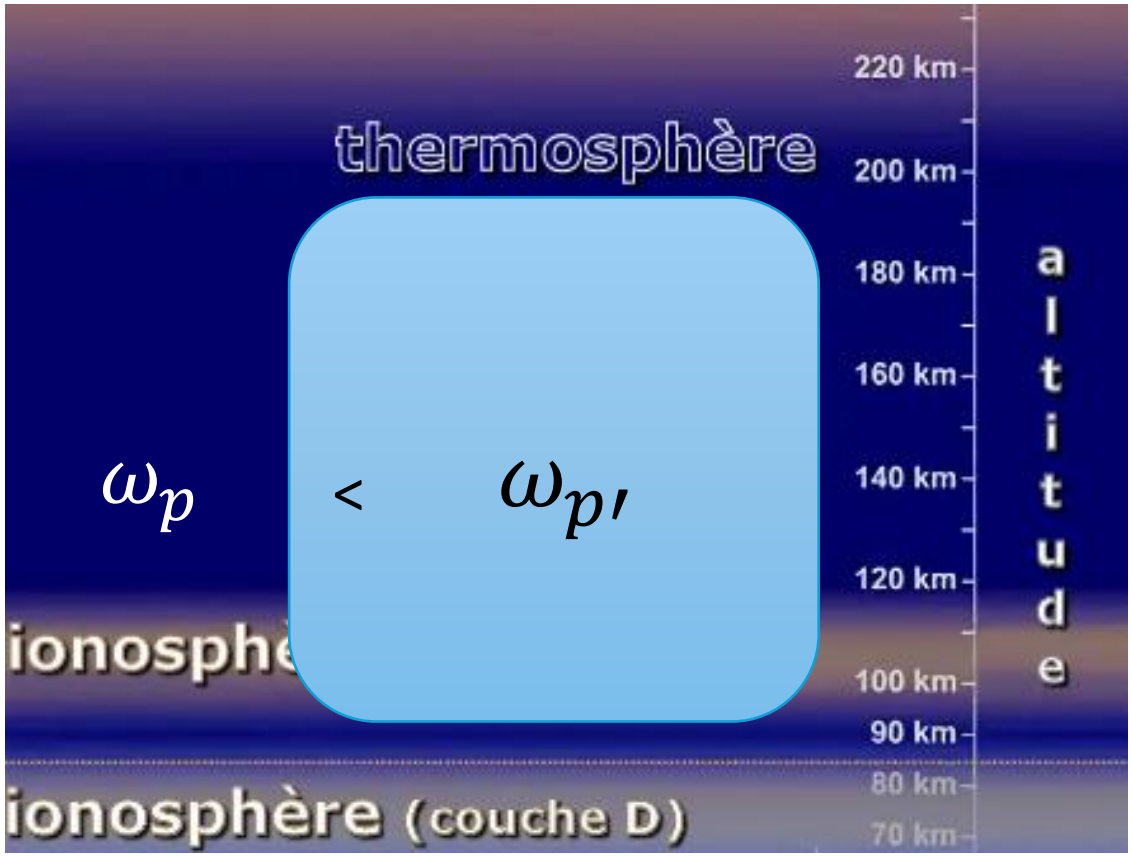
↗ de la pulsation plasma ω_p

Conséquences



Source : aquaportail.com

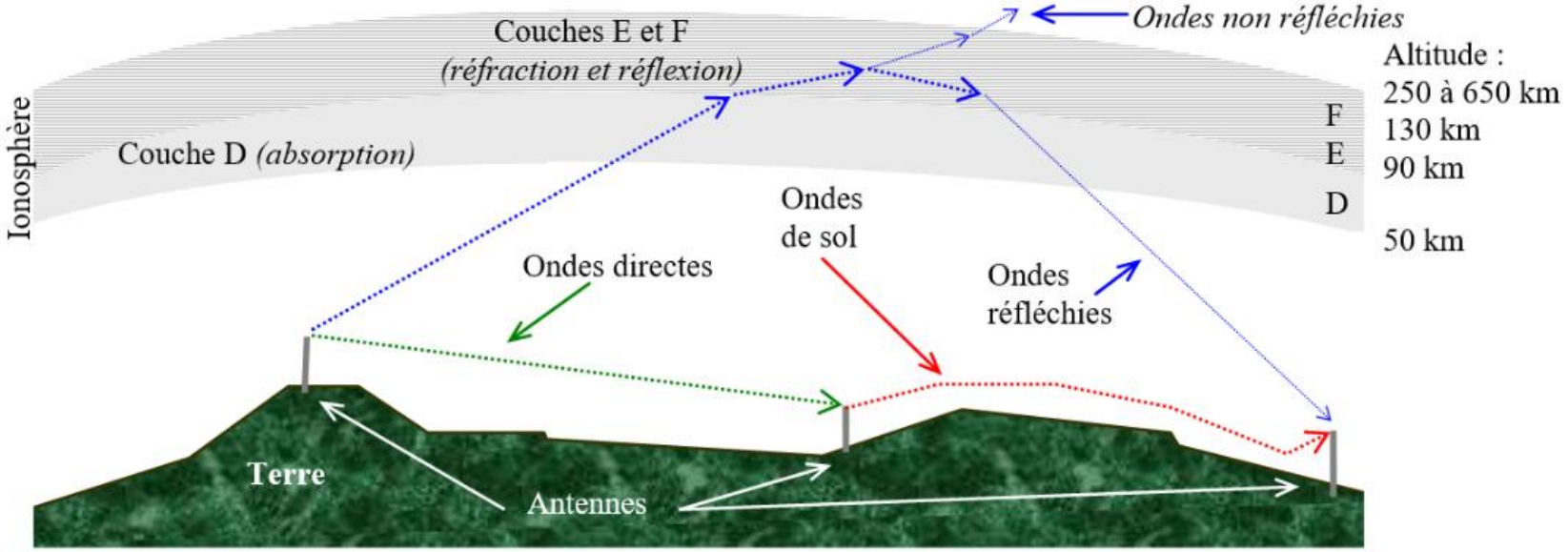
Conséquences



Apparition de nuages ionisés

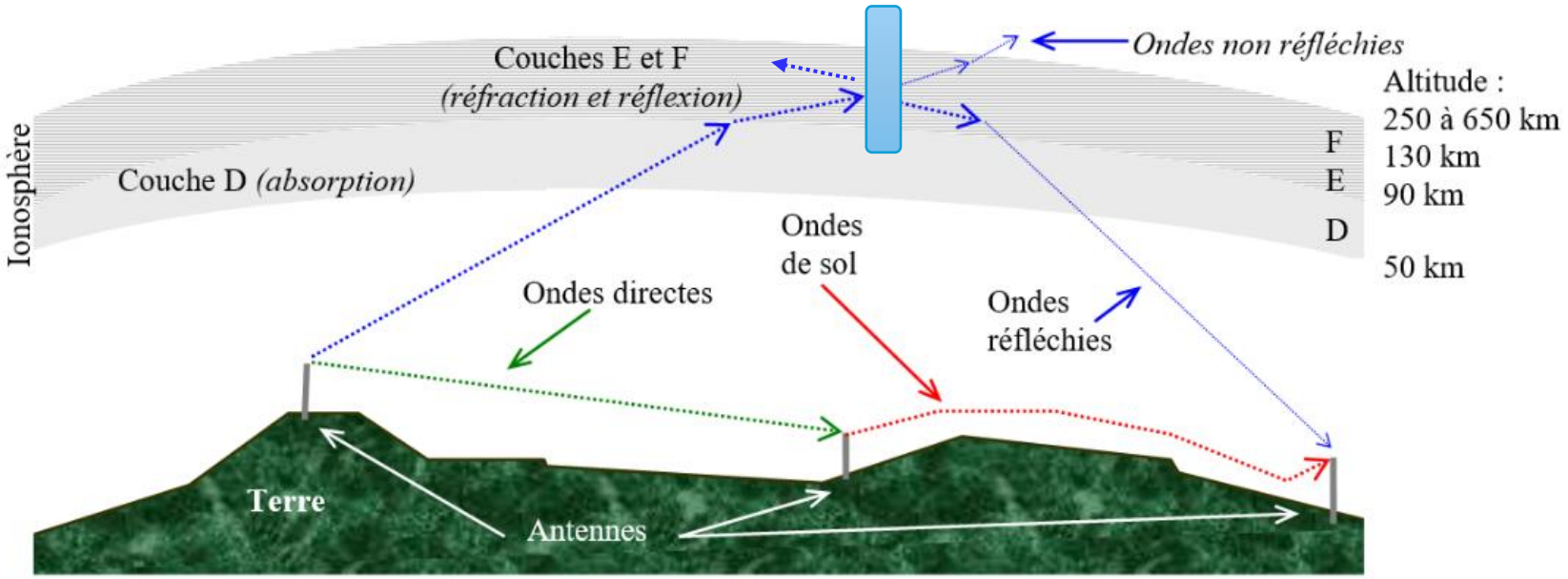
Source : aquaportail.com

Problème dans la propagation des ondes HF



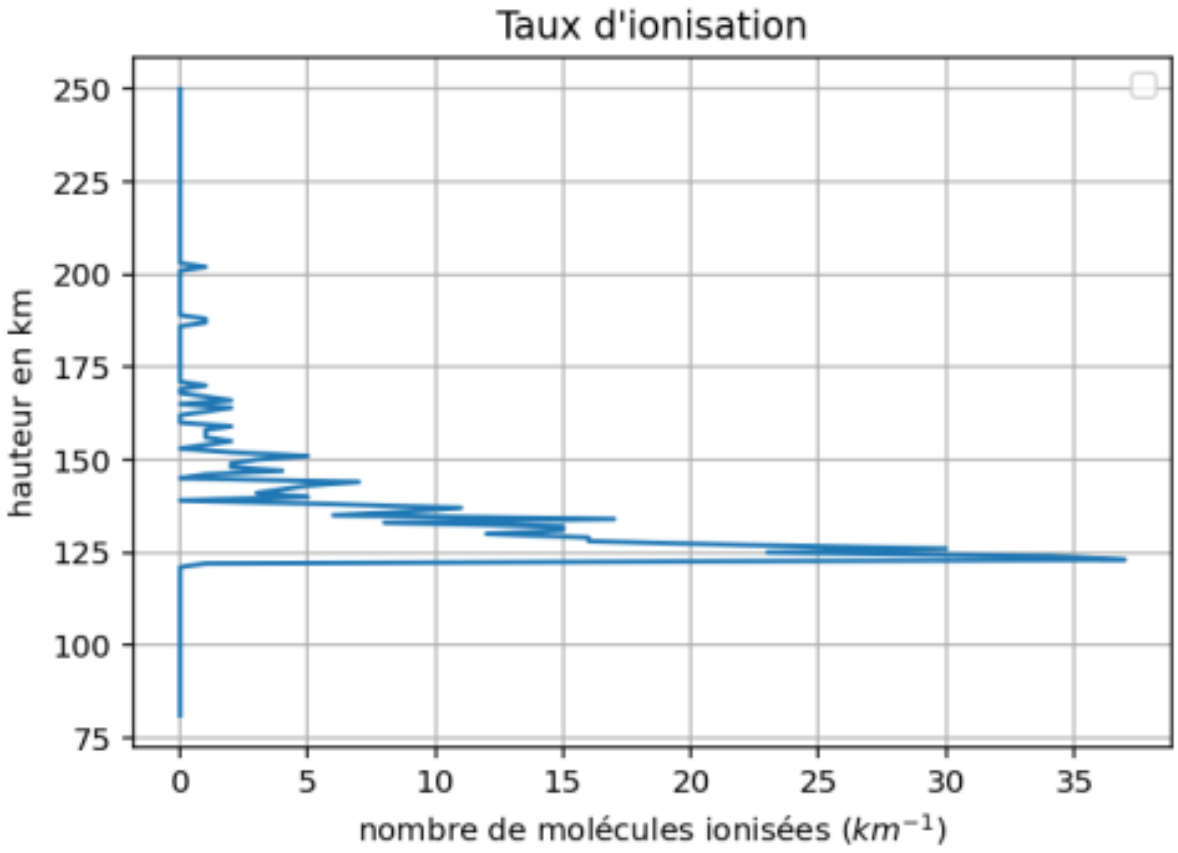
Source : rogerbeep.fr

Problème dans la propagation des ondes HF



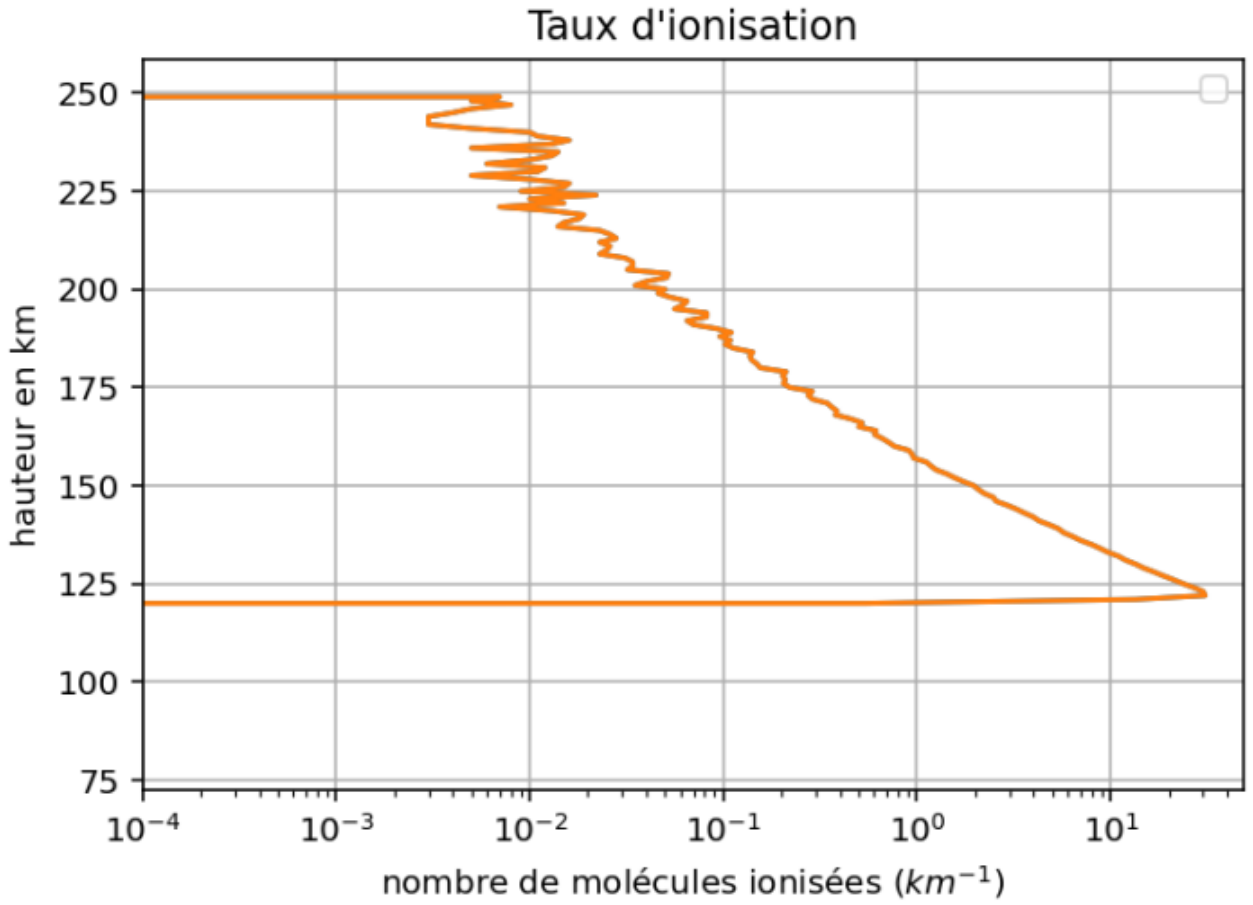
Exploitation des résultats

Nombre de molécules ionisées par km causé par un électron

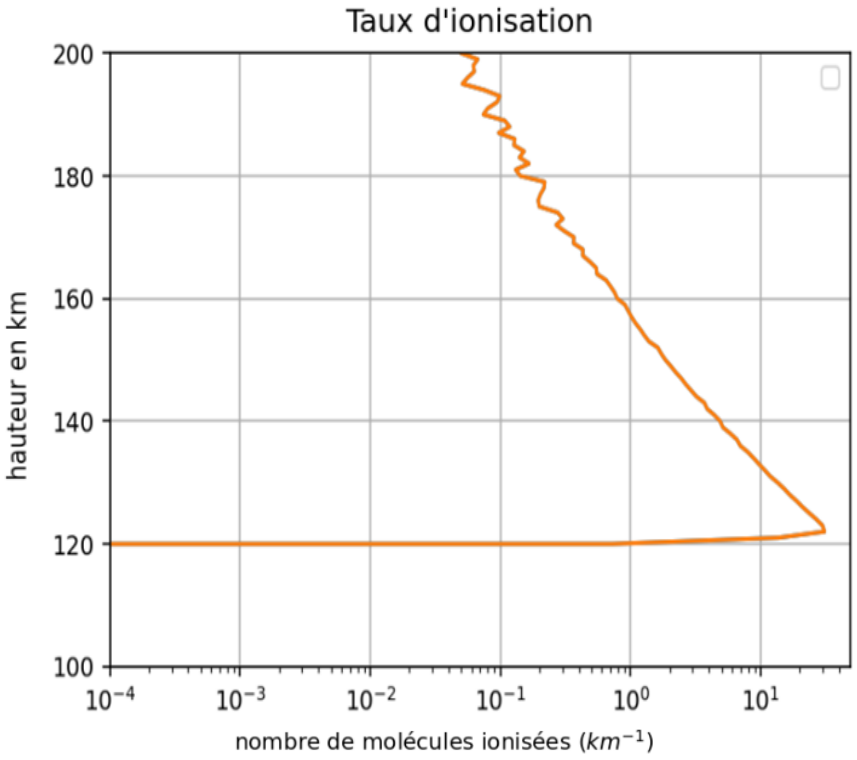


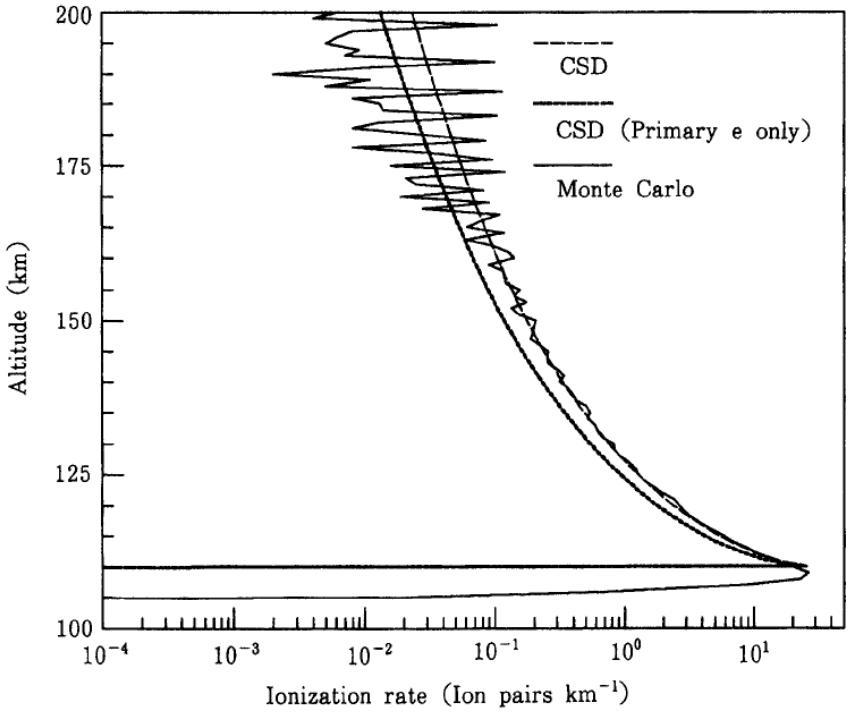
Exploitation des résultats

Nombre de molécules ionisées par km,
la moyenne sur mille électrons



Comparaison des résultats avec d'autres études menées

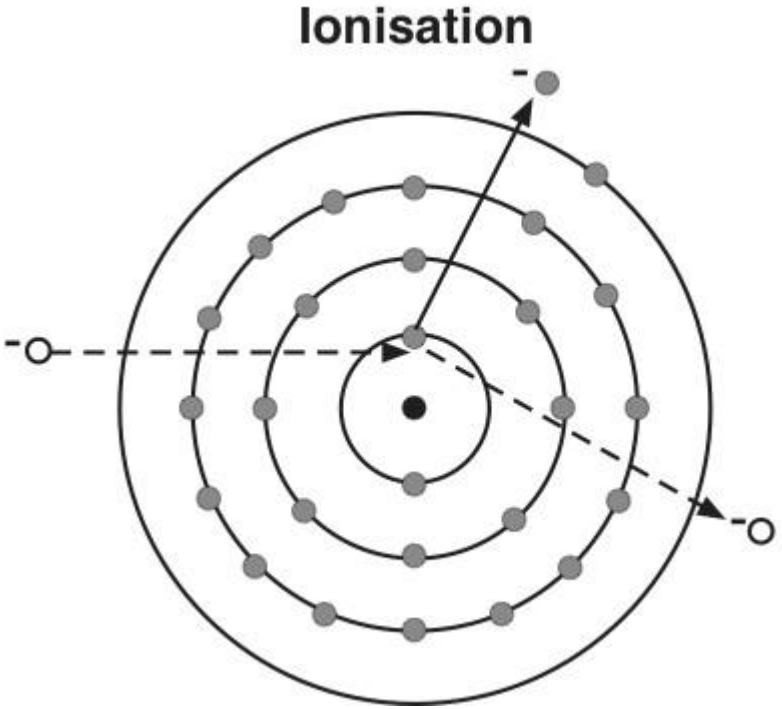




Source : Monte Carlo calculation of ionization and excitation Rates

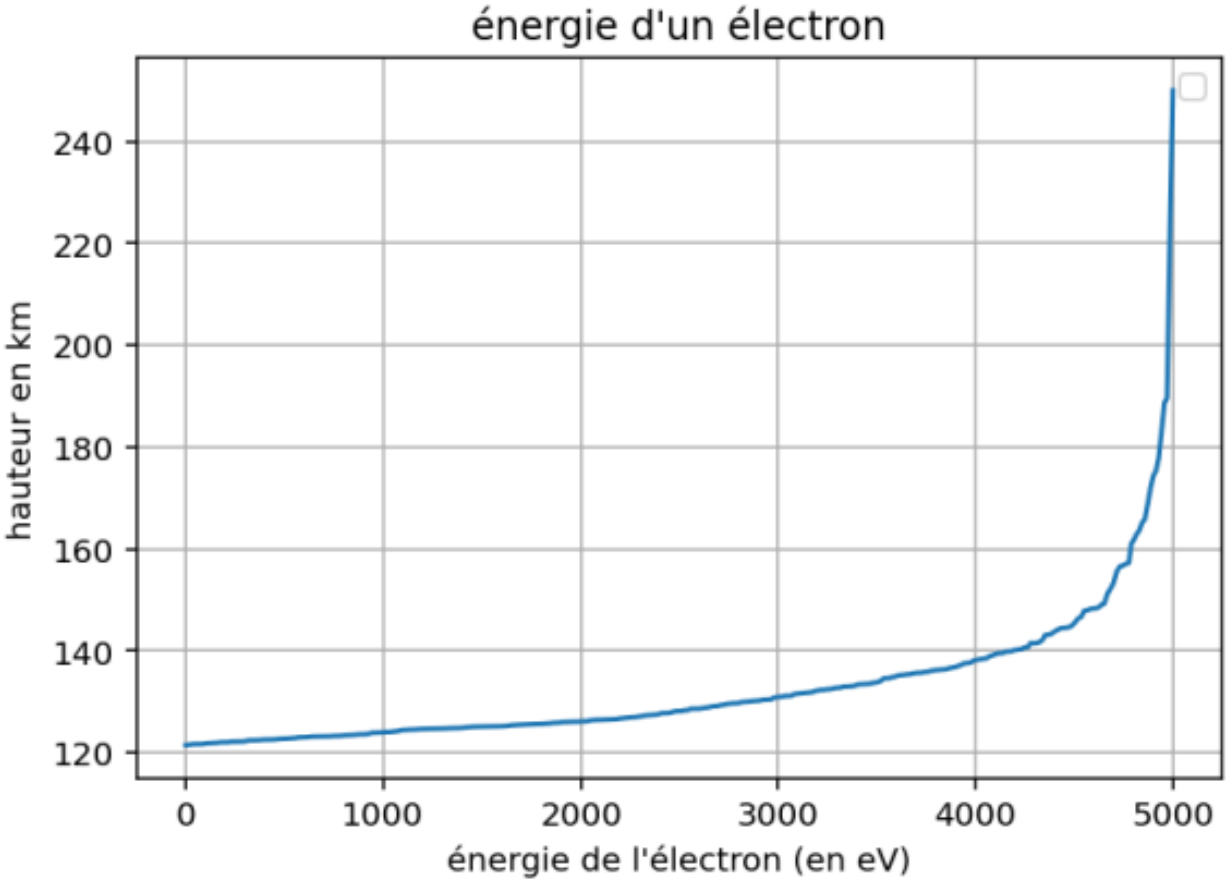
Limites de la modélisation

Pas de calcul des électrons secondaires

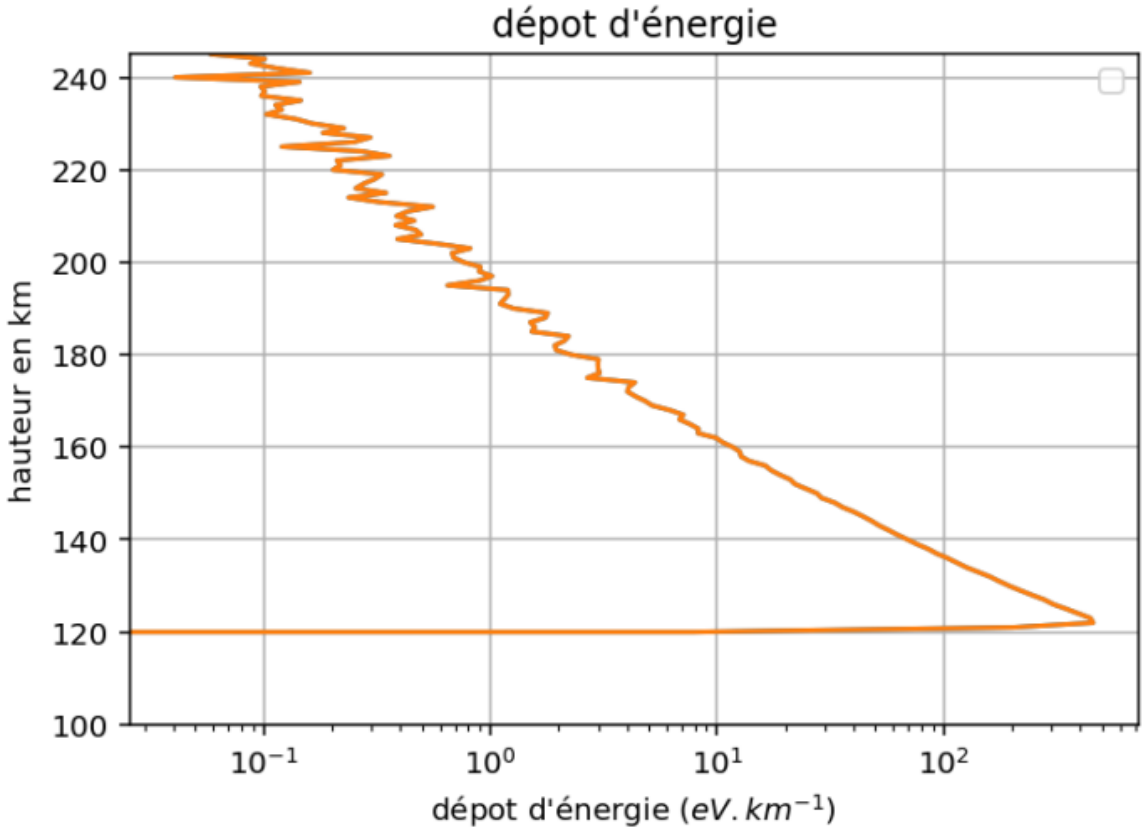


Source : Radioprotection en radiodiagnostic – 3e édition

Variation de l’énergie d’un électron
en fonction de l’altitude



Dépôt d'énergie (moyenne pour 1000 électrons)



Conclusion

Merci pour votre attention



Annexe 1

Caractérisation du processus de Poisson pour une variable aléatoire N_l :

$$I. \forall\, t_o = 0 < l_1 < \cdots < l_k, \qquad les\ v.a.\ \left(N_{l_k} - N_{l_{k-1}}\right), \ldots, \left(N_{l_1} - N_{l_0}\right)\ sont\ indépendantes$$

$$II.\ P(N_{l+h} - N_l = 1) = \mu h + o(h)\ quand\ h \to 0^+, \text{à } l\ fixé$$

$$III.\ P(N_{l+h} - N_l > 1) = o(h)\ quand\ h \to 0^+, \text{à } l\ fixé$$

Si N_l vérifie ces conditions, alors elle suit une loi de Poisson de paramètre μ

Annexe 2

Données initiales utilisées

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad
import random as rd


R = 8.314       #constantes des gaz parfaits
g=9.817         #accéleration de la pesanteur
k= 1.38e-23         #constante de Boltzmann

#rayon atomique
r_O = 66e-12
r_O2 = 146e-12
r_N2 = 158e-12

#O
O_E_ioni = 13.6   #en eV
O_E_exc = 9.43


#O2
O2_E_ioni = 12.1   #en eV
O2_E_exc = 9.7

#N2
N2_E_ioni =  15.6
N2_E_exc = 13
```

```python
#composition de l'atmosphère
def compo_atm(h):

    if h < 100 :
        prop_N2 = 78
        prop_O2 = 22
        prop_O = 0
    else :
        prop_N2 = -0.38*h + 116
        prop_O2 = -0.14*h + 36
        prop_O = 0.57*h - 57
    return (prop_O,prop_O2,prop_N2)



#profil de la temperature en K
def temperature(h):
    if 51 <= h < 71:
        return 270.7 - 2.8 * (h - 51)
    elif 71 <= h < 85:
        return 214.7 - 2.0 * (h - 71)
    elif 85 <= h < 91:
        return 186.9
    elif 91 <= h <= 250:
        return 186.9 + 3.67 * (h - 91)
```

Annexe 3

Modélisation de l'atmosphère

```python
#pression à l'altitude h en Pa
def Pression (h):

    def f(z):
        return (l(z) * g * massemol_eff(z)) / (R * temperature(z))

    if 51 <= h < 71 :
        res, err = quad(f, h, 51)
        return 70.5 * np.exp(res * 10**3)

    elif 71 <= h < 85:
        res, err = quad(f, h, 71)
        return 4.48 * np.exp(res * 10**3)


    elif 85 <= h < 91 :
        res, err = quad(f, h, 85)
        return 0.446 * np.exp(res * 10**3)

    elif 91 <= h <= 250:
        res, err = quad(f, h, 91)
        return 0.154 * np.exp(res * 10**3)
```

```python
#masse molaire effective de l'air en kg/mol
def massemol_eff (h):
    compo = compo_atm(h)
    return compo[0]*16e-5 + compo[1]*32e-5 + compo[2]*28e-5


#facteur correctif pour g
def l(h):
    return ( 6378 / (6378 + h) )**2



#expression du libre parcours moyen en km
def libre_parcours_moy(h):
    compo = compo_atm(h)
    d = (r_O * compo[0] + r_O2 * compo[1] + r_N2 * compo[2]) * 10**-2
    return (k*temperature(h) * 10**-3) / (np.pi * Pression(h) * d**2)
```

Annexe 3

Code de la modélisation du parcours de l'électron

```
#def de la fonction factorielle
def fact(n):
    if n == 0:
        return 1
    else:
        return n  * fact(n-1)



#def de la probabilité de Poisson
def P(k, mu):
    return np.exp(-mu) * mu**k/ fact(k)


#simule le parcours d'un seul électron
def parcours_electron (h, E):
    compteur_coll =[]                          #nombre de collision
    ha = h
    E_elec = E
    c_ioni = []
    Tabl = [[ha], [E]]           #Tabl[0] = hauteur, Tabl[1] = énergie à h

    while ha > 80 and E_elec > 9.43 :
        lambd = libre_parcours_moy(ha)
        dist_parc =  min (lambd, 5)
        H_coll = nb_coll(ha, dist_parc)
        if len(H_coll) != 0 :
            H_coll.sort(reverse = True )
            while len(H_coll) != 0:
                compteur_coll.append(H_coll[0])
                E_elec, c_ioni = ener_coll(E_elec, H_coll[0], c_ioni)
                Tabl[0].append(H_coll[0])
                Tabl[1].append(E_elec)
                H_coll.pop(0)


        ha = ha - dist_parc


    return compteur_coll, Tabl, c_ioni
```

```
#donne les hauteurs auquels il ya collision
def nb_coll(h, dist_parc):
    lambd = libre_parcours_moy(h)
    mu = dist_parc / lambd
    r=rd.random()
    hc = []
    if r < P(0,mu) :
        return hc
    elif r < P(0,mu) + P(1,mu) :                # une collision
        f = rd.random()              #pour déterminer hauteur de la collision
        hc.append(h - f*dist_parc)
        return hc

    elif r < P(0,mu) + P(1,mu) +P(2,mu) :        # deux collisions
        for i in range (2):
            f = rd.random()
            hc.append(h - f*dist_parc)
        return hc

    elif r < P(0,mu) + P(1,mu) +P(2,mu)+P(3,mu) :        # trois collisions
        for i in range (3):
            f = rd.random()
            hc.append(h - f*dist_parc)
        return hc

    elif r < P(0,mu) + P(1,mu) +P(2,mu)+P(3,mu)+ P(4,mu) :  # quatre collisions
        for i in range (4):
            f = rd.random()
            hc.append(h - f*dist_parc)
        return hc
    return hc
```

Code de la modélisation de la collision

```
c_exc = 0           #compteur d'atome excité


def atome_collision (h):
    #renvoie avec quelle espèce l'électron est rentré en contact
    # 1 : O      2 : O2       3 : N2
    P = rd.randint(0, 100)
    if h < 100 :
        if P > 22 :
            return (3)
        else :
            return (2)
    else :
        compo = compo_atm(h)
        if P < compo[0]:
            return(1)
        elif P < compo[0] + compo[1] :
            return(2)
        elif P < compo[0] + compo[1] +compo[2] :
            return(3)
```

```
def ener_coll (Ei, h, c_ioni):
    #renvoie la nature de la collision, la nouvelle énergie de l'électron
    atome = atome_collision (h)
    global c_exc
    if atome == 1 :
        if Ei > O_E_ioni :
            c_ioni.append(h)
            Ef = Ei - O_E_ioni
        elif O_E_exc < Ei :
            c_exc += 1
            Ef = Ei - O_E_exc
        else : Ef = Ei




    elif atome == 2 :

        if Ei > O2_E_ioni :
            c_ioni.append(h)
            Ef = Ei - O2_E_ioni
        elif O2_E_exc < Ei :
            c_exc += 1
            Ef = Ei - O2_E_exc
        else : Ef = Ei

    elif atome == 3 :

        if Ei > N2_E_ioni :
            c_ioni.append(h)
            Ef = Ei - N2_E_ioni
        elif N2_E_exc < Ei :
            c_exc += 1
            Ef = Ei - N2_E_exc
        else : Ef = Ei

    return (Ef, c_ioni)
```

Annexe 5

Exploitation des résultats : calcul du taux d'ionisation et du dépôt d'énergie

```python
H= [ h for h in range (250, 80, -1)]


#calcul du dépot d'énergie en eV/km
def depot_ener(h, E) :
    a, Tabl, c_ioni = parcours_electron(250, 5000)
    n = len(Tabl[1])
    depot = [0 for i in H]

    for i in range(len(H)) :
            while Tabl[0][0] > H[i] and len(Tabl[0])!=1 :
                depot[i]+= Tabl[1][0] -Tabl[1][1]
                Tabl[0].pop(0)
                Tabl[1].pop(0)
    return depot




def moy_depot (h, E):
    t_tot = []
    for i in range (1000):
        t_tot.append(depot_ener(250, 5000))
    n = len(t_tot[0])
    t=[0 for i in range (n)]


    for i in range (n):
        for j in range(1000):
            t[i] += t_tot[j][i]
        t[i] = t[i]* (1/1000)

    return (t, t_tot)
```

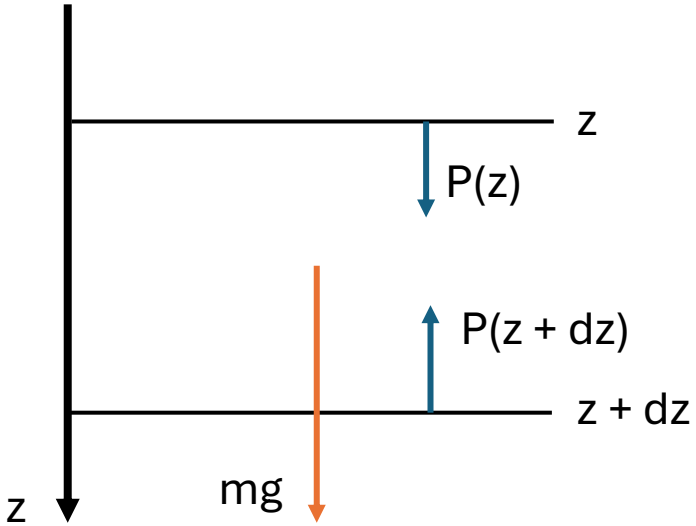
```python
#taux d'ionisation pour un seul électron
def taux_ioni(h, E):
    a, b, cio =  parcours_electron(h, E)
    c_io = cio.copy()
    ioni_rate = [0 for i in H]
    for i in range(len( H)):
        while c_io[0] >= H[i] and len(c_io)!=
            ioni_rate[i] +=1
            c_io.pop(0)
    return ioni_rate


#moyenne pour 1000 électrons
def moy1000 (h, E):
    t_tot = []
    for i in range (100):
        t_tot.append(taux_ioni(250, 5000))
    n = len(t_tot[0])
    t=[0 for i in range (n)]


    for i in range (n):
        for j in range(100):
            t[i] += t_tot[j][i]
        t[i] = t[i]* (1/100)

    return (t, t_tot)
```

Equilibre hydrostatique



On applique le principe fondamental de la dynamique à la tranche de gaz à l'équilibre :

$$0 = mg + SP(z) - SP(z + dz)$$

$$0 = mg - S \cdot dz \, \frac{dP(z)}{dz}$$

$$\Leftrightarrow \qquad \frac{dP(z)}{dz} = \mu g$$

Pulsation plasma

En appliquant le principe fondamental de la dynamique à l'électron non relativiste soumis au champ $\vec{E}$, il vient :

$$-e\vec{E} = m_e \frac{d\vec{v}}{dt} = im_e\omega\vec{v}$$

D'où $$\vec{j} = \rho\vec{v} = -i\frac{ne^2}{m_e\omega}\vec{E}$$

$$\overrightarrow{rot}\,\vec{B} = \left(\mu_0\vec{j} + \frac{i\omega}{c^2}\right)\vec{E} \quad \Longrightarrow \quad \Delta\vec{E} - \left(-\frac{\mu_0 ne^2}{m_e} + \frac{\omega^2}{c^2}\right)\vec{E} = 0$$

$$\omega_p{}^2 = \frac{ne^2}{m_e\varepsilon_0}$$